Qassim University جامعة القصيم

Semester 443 - 2023, 1444

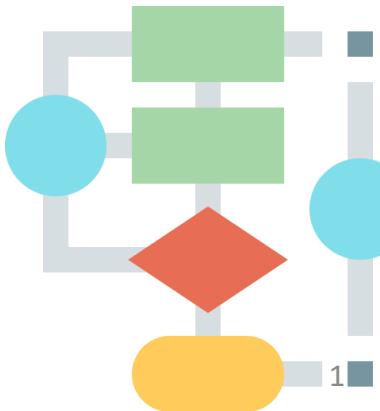
Course CS315 Algorithms Analysis and Design

B-Trees

Lecture 6 & 7

Prof. Dr Mohammed Al-Hagery

Supervisordr@yahoo.com

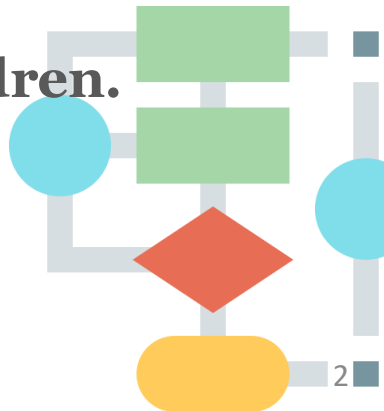


Introduction to B-Tree

- B-trees are balanced search tree.

Basically, only two children for each node in the B-Tree, although More than two children are possible.

- B-Tree, stores all information in the leaves and stores only keys and Child pointer.
- All leaves are at the same depth in the tree.
- If an internal B-tree node x contains n[x] keys then x has n[x]+1 children.



Difference Between B-tree and Binary tree

There are two types of non-linear data structures namely,

B-Tree and **Binary Tree**. These two terms sound similar but they are absolutely different from each other. The most basic difference between a B-Tree and a Binary Tree is that a B-Tree is used for data storage on a disk. **B-Tree** is also called Balanced Sort Tree.

whereas a Binary Tree is used for data storage in RAM.

Example of B-Tree

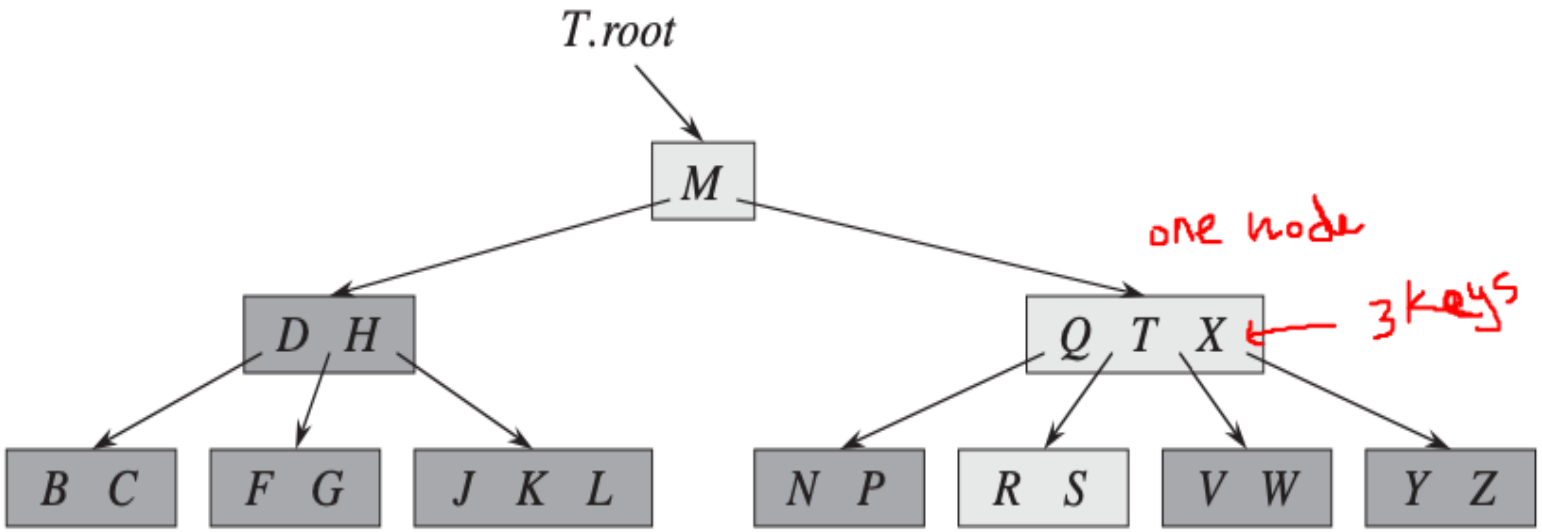


Figure 18.1 A B-tree whose keys are the consonants of English. An internal node x containing $x.n$ keys has $x.n + 1$ children. All leaves are at the same depth in the tree. The lightly shaded nodes are examined in a search for the letter R.

Example of B-Tree

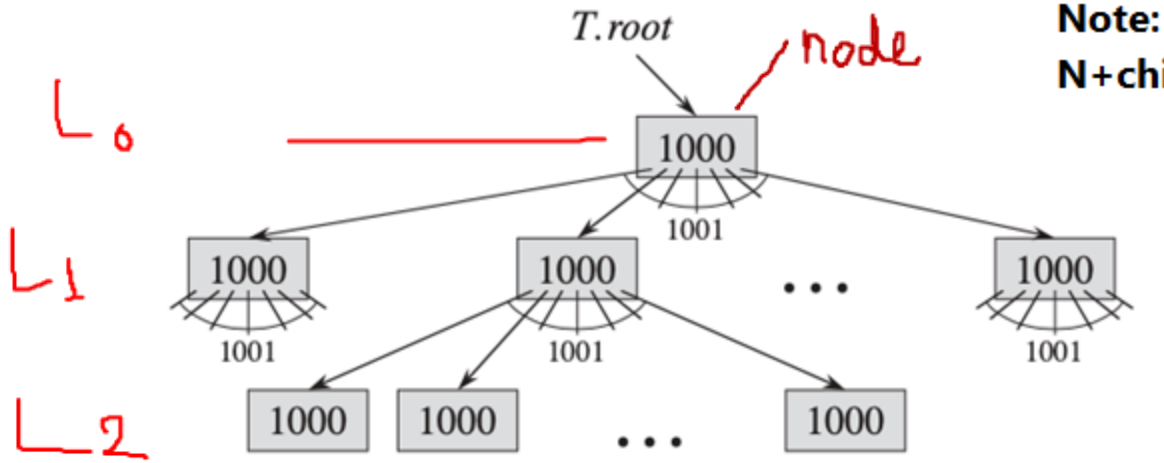


Figure 18.3 A B-tree of height 2 containing over one billion keys. Shown inside each node x is $x.n$, the number of keys in x. Each internal node and leaf contains 1000 keys. This B-tree has 1001 nodes at depth 1 and over one million leaves at depth 2.

Application of B-Tree

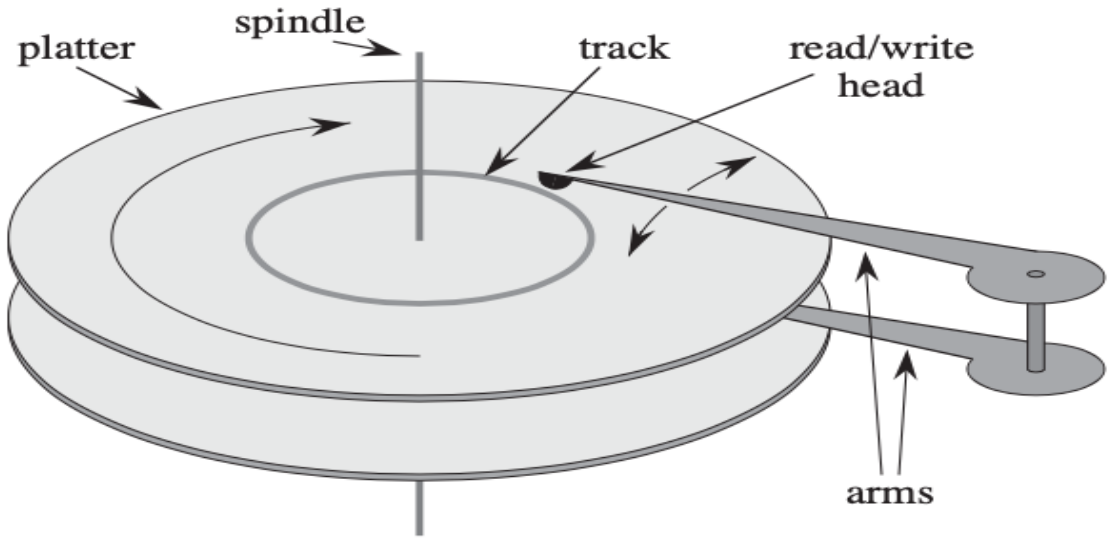


Figure 18.2 A typical disk drive. It comprises one or more platters (two platters are shown here) that rotate around a spindle. Each platter is read and written with a head at the end of an arm. Arms rotate around a common pivot axis. A track is the surface that passes beneath the read/write head when the head is stationary.

It designed to work on magnetic disks or other (direct access) secondary storage devices.

Properties of B-Tree

A B-Tree T is a rooted tree, having the following properties:

1. Every node x has the following fields:

*** n[x]: is the no. of keys currently stored in node x**

*** The n[x] keys themselves, stored in non-decreasing order, so that**

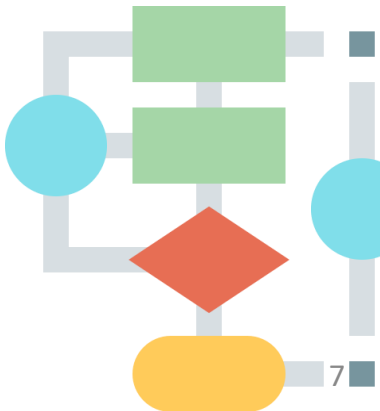
key1[x] ≤ key2[x]…… ≤keyn-1[x] ≤ keyn[x].

*** Leaf[x], a Boolean value that is TRUE if x is a leaf and FALSE if x is an internal node.**

2. Each internal node x also contains n[x]+1 pointers (Childs) c1[x], c2[x],---------cn[x]+1[x].

Note: Number of child's =Numb of Keys (in the root) +1.

3. All leaves have the same depth, which is the tree's height h.



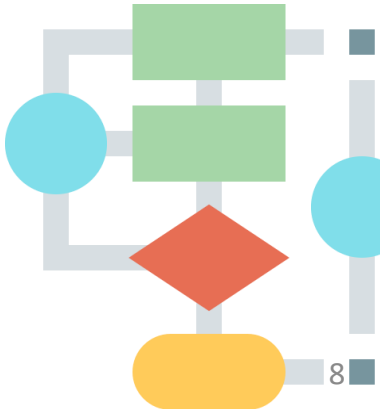
Properties of B-Tree

4. There are lower and upper bounds on the no. of keys. A node can contain: these bounds, which can be expressed in terms of a fixed integer t≥2 called the minimum degree of B-Tree (the number of Childs must be 2 or more).

* Every node other than the root must have at least t-1 keys, then root has at least t children if the tree is non empty, the root must have at least one key.
* Every node can contain at most 2t-1 keys. Therefore, an internal node can have most 2t children and 2t-1 keys, **so** we can say that a node is full in this case.

childs

keys(n)



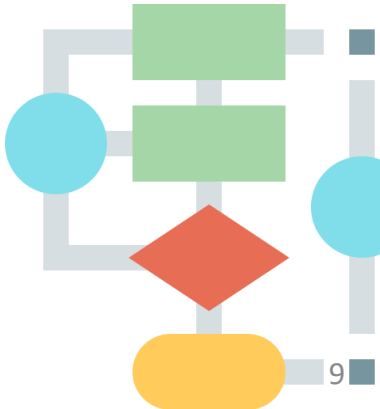
Basic operation on B-tree

1 - B-TREE-SEARCH :- Searching in B.Tree

2 - B-TREE-INSERT :- Inserting key in B Tree

3 - B-TREE-CREATE :- Creating a B Tree

4 -B-TREE-DELETE :- Deleting a key from B.Tree

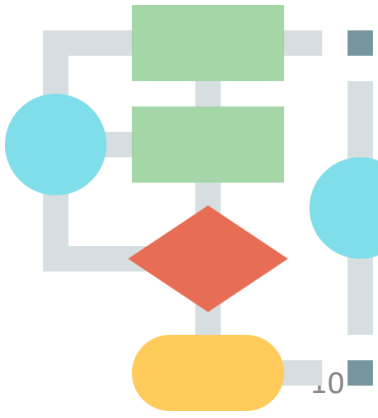


X is a subtree and k is searching element

```
B-TREE-SEARCH(x, k)
1  i = 1
2  while i ≤ x.n and k > x.key_i
3       i = i + 1
4  if i ≤ x.n and k == x.key_i
5       return (x, i)
6  elseif x.leaf
7       return NIL
8  else DISK-READ(x.c_i)
9       return B-TREE-SEARCH(x.c_i, k)
```

Complexity=O(t)

Complexity= $O(\log_t n)$

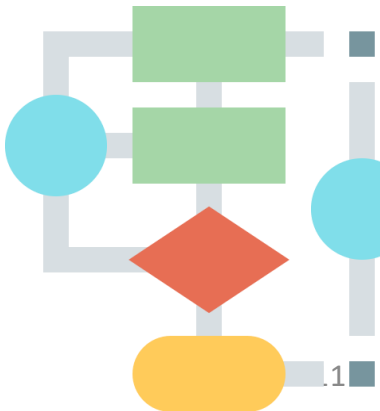


Creating an empty B tree

B-TREE-CREATE(T)

1 x = ALLOCATE-NODE()
2 $x.leaf$ = TRUE
3 $x.n = 0$
4 DISK-WRITE(x)
5 $T.root = x$

B-TREE-CREATE requires $O(1)$ disk operations and $O(1)$ CPU time.



Insert

Cannot just create a new leaf node and insert it

- resulting a tree, which is not B-tree

Insert new key into an existing leaf node

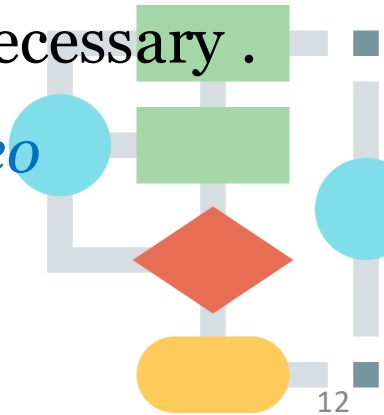
If leaf node is full *(no of keys=n and no of child's =n+1)*

- Split full node **y** (with 2t-1) key around its median key t[y] into two nodes each having t-1 keys.
- Move the median key y's parents. الاوسط

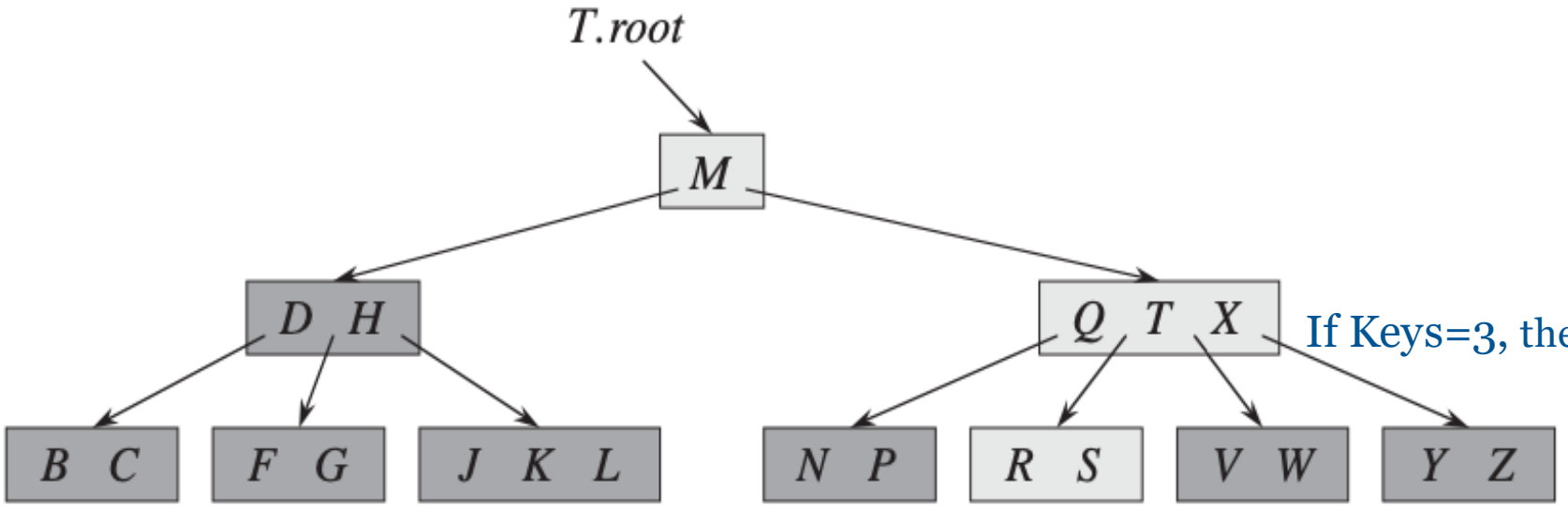
- If parents is full , recursively split , all the way to the root node if necessary .

- If root is full , split root, height of tree increase by one . *See the video*

https://www.youtube.com/watch?v=aZjYr87r1b8



Insert



If Keys=3, then max=Childs=4

Figure 18.1 A B-tree whose keys are the consonants of English. An internal node x containing $x.n$ keys has $x.n + 1$ children. All leaves are at the same depth in the tree. The lightly shaded nodes are examined in a search for the letter R.

الحروف الساكنه
غير المتحركه

Split

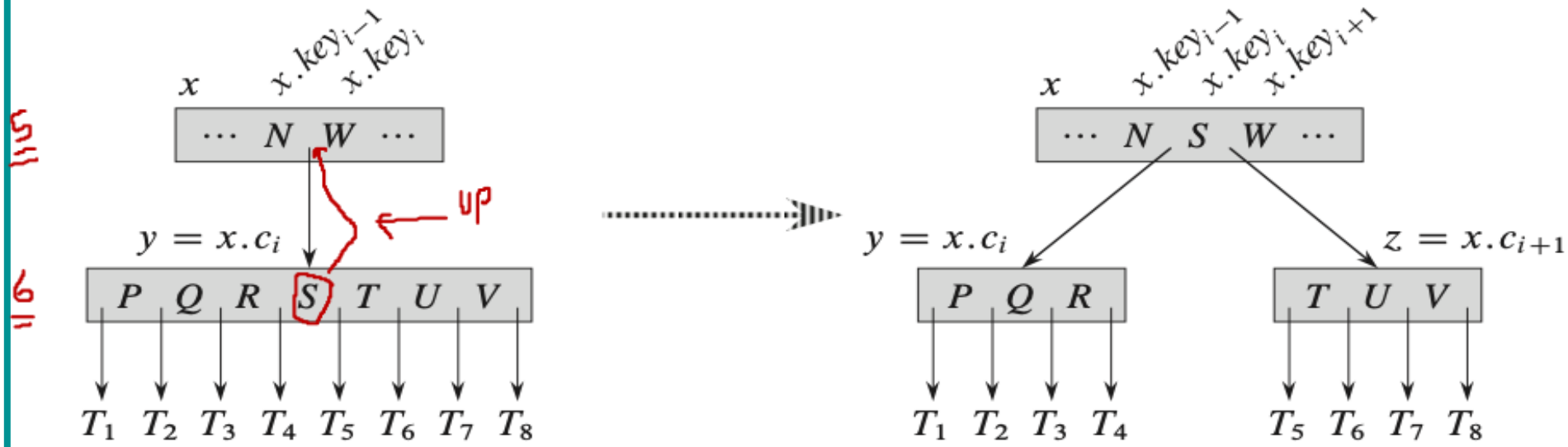
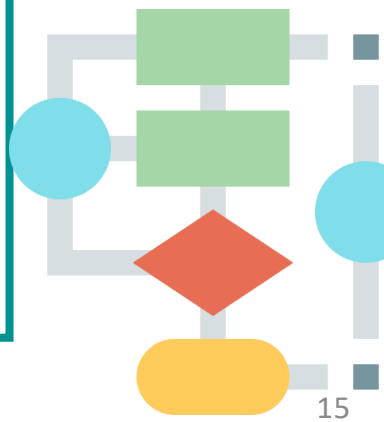


Figure 18.5 Splitting a node with $t = 4$. Node $y = x.c_i$ splits into two nodes, y and z, and the median key S of y moves up into y's parent.

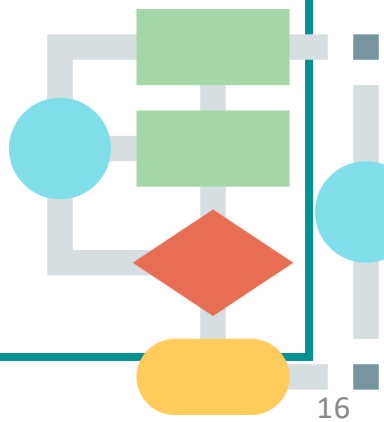
B-TREE-SPLIT-CHILD ALGORITHM

```
B-TREE-SPLIT-CHILD(x, i)
 1  z = ALLOCATE-NODE()
 2  y = x.c_i
 3  z.leaf = y.leaf
 4  z.n = t − 1
 5  for j = 1 to t − 1
 6       z.key_j = y.key_{j+t}
 7  if not y.leaf
 8       for j = 1 to t
 9            z.c_j = y.c_{j+t}
10  y.n = t − 1
11  for j = x.n + 1 downto i + 1
12       x.c_{j+1} = x.c_j
13  x.c_{i+1} = z
14  for j = x.n downto i
15       x.key_{j+1} = x.key_j
16  x.key_i = y.key_t
17  x.n = x.n + 1
18  DISK-WRITE(y)
19  DISK-WRITE(z)
20  DISK-WRITE(x)
```



B-TREE-INSERT ALGORITHM

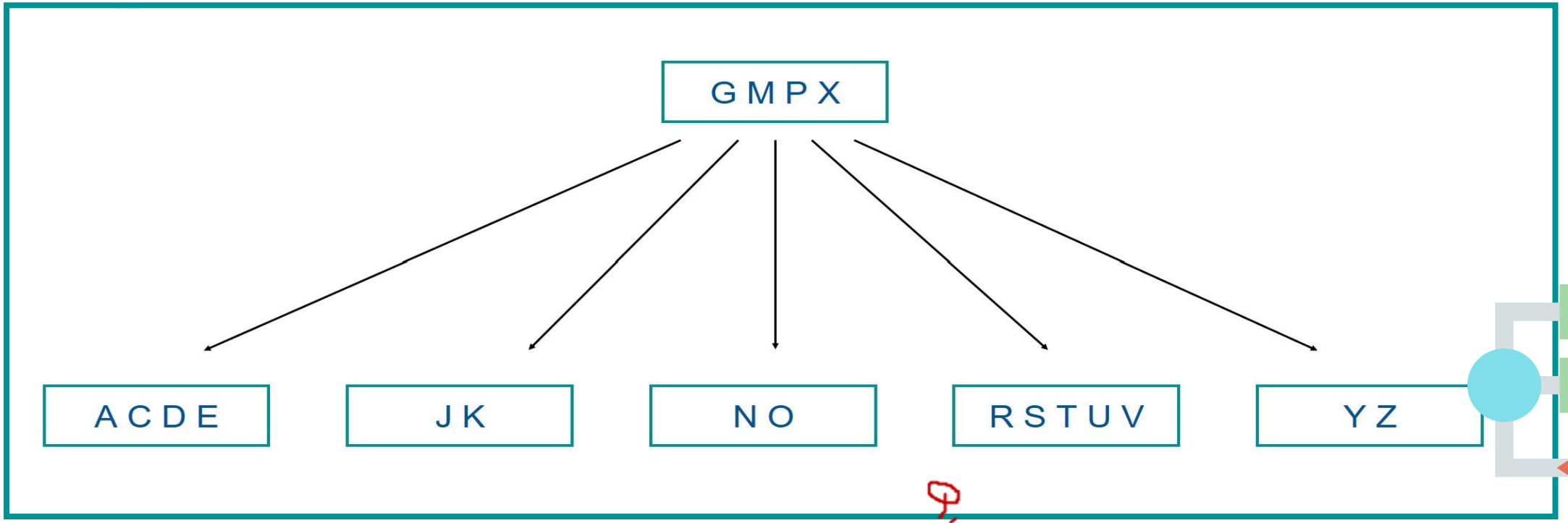
```
B-TREE-INSERT(T, k)
 1  r = T.root
 2  if r.n == 2t − 1
 3       s = ALLOCATE-NODE()
 4       T.root = s
 5       s.leaf = FALSE
 6       s.n = 0
 7       s.c_1 = r
 8       B-TREE-SPLIT-CHILD(s, 1)
 9       B-TREE-INSERT-NONFULL(s, k)
10  else B-TREE-INSERT-NONFULL(r, k)
```



Can also insert in a single pass down the tree , instead of going down during search then recursively splitting on the way up.

- Split full nodes encountered on the way down during search .

Example



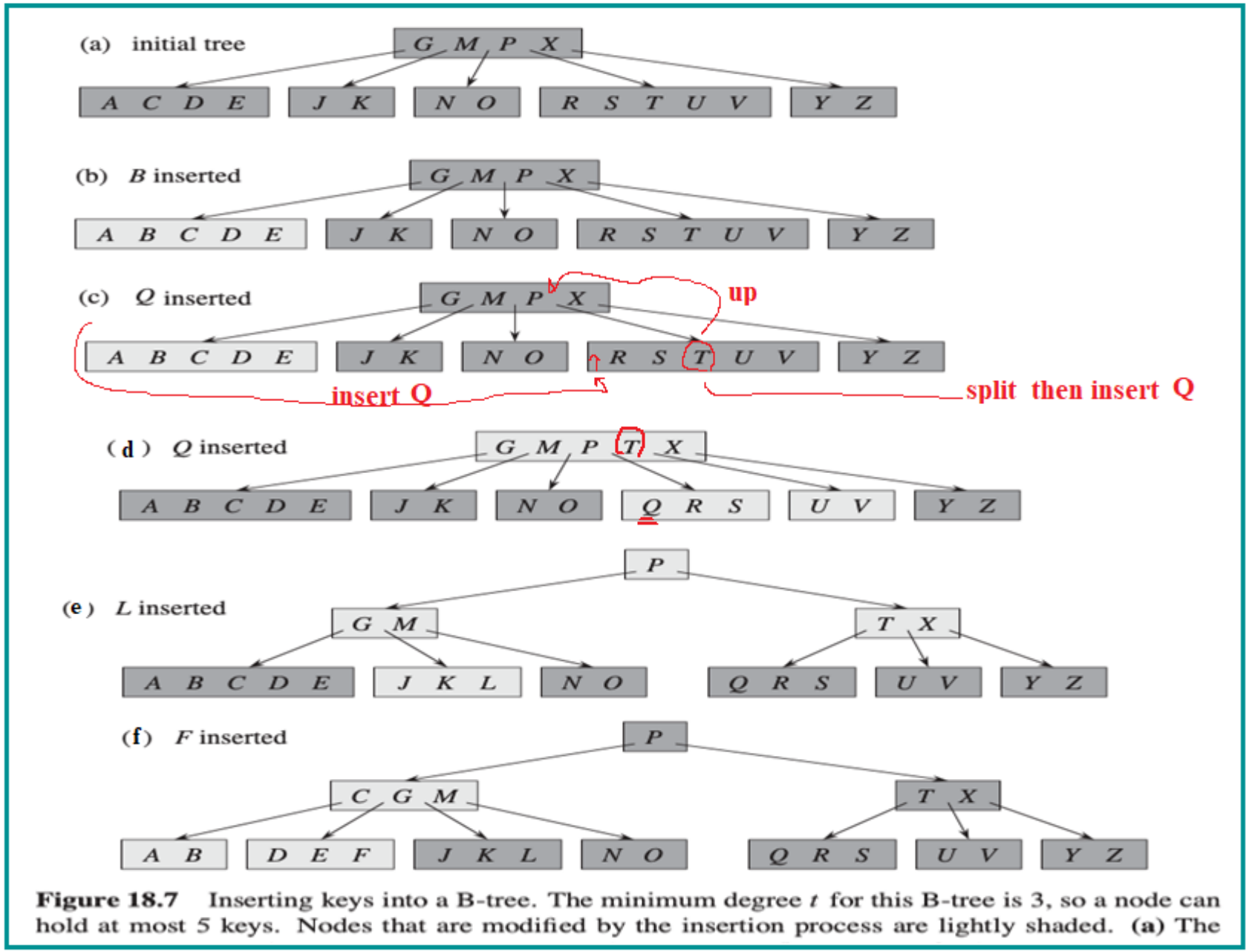
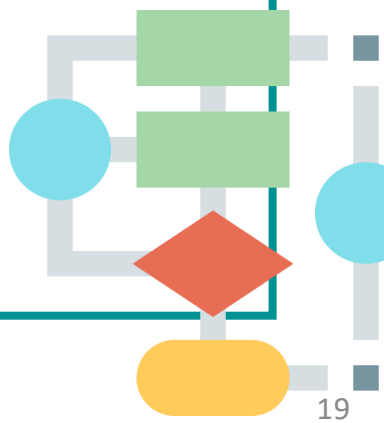


Figure 18.7 Inserting keys into a B-tree. The minimum degree t for this B-tree is 3, so a node can hold at most 5 keys. Nodes that are modified by the insertion process are lightly shaded. **(a)** The

In c) there are two processes for Q: split then move the median up to the root then insert Q. according to the sequence.

Insert non full B tree

```
B-TREE-INSERT-NONFULL(x, k)
 1  i = x.n
 2  if x.leaf
 3       while i ≥ 1 and k < x.key_i
 4            x.key_{i+1} = x.key_i
 5            i = i − 1
 6       x.key_{i+1} = k
 7       x.n = x.n + 1
 8       DISK-WRITE(x)
 9  else while i ≥ 1 and k < x.key_i
10            i = i − 1
11       i = i + 1
12       DISK-READ(x.c_i)
13       if x.c_i.n == 2t − 1
14            B-TREE-SPLIT-CHILD(x, i)
15            if k > x.key_i
16                 i = i + 1
17       B-TREE-INSERT-NONFULL(x.c_i, k)
```

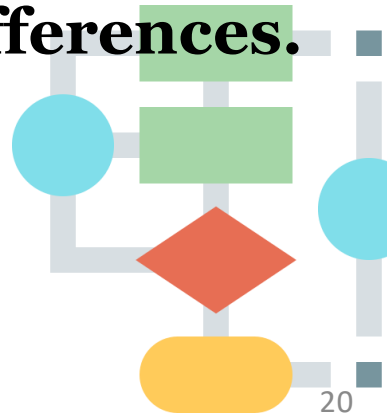


Exercise in Inserting a B-Tree

- Insert the following keys in B-tree when t=3 is (the minimum degree 3) :

3, 7, 9, 23, 45, 1, 5, 14, 25, 24, 13, 11, 8, 19, 4, 31, 35, 56

- Check your approach with a neighbour and discuss any differences.

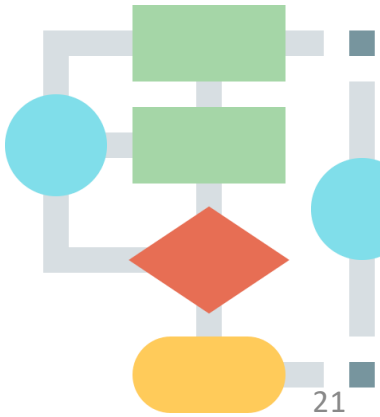


Deleting from B-Trees

The Concept

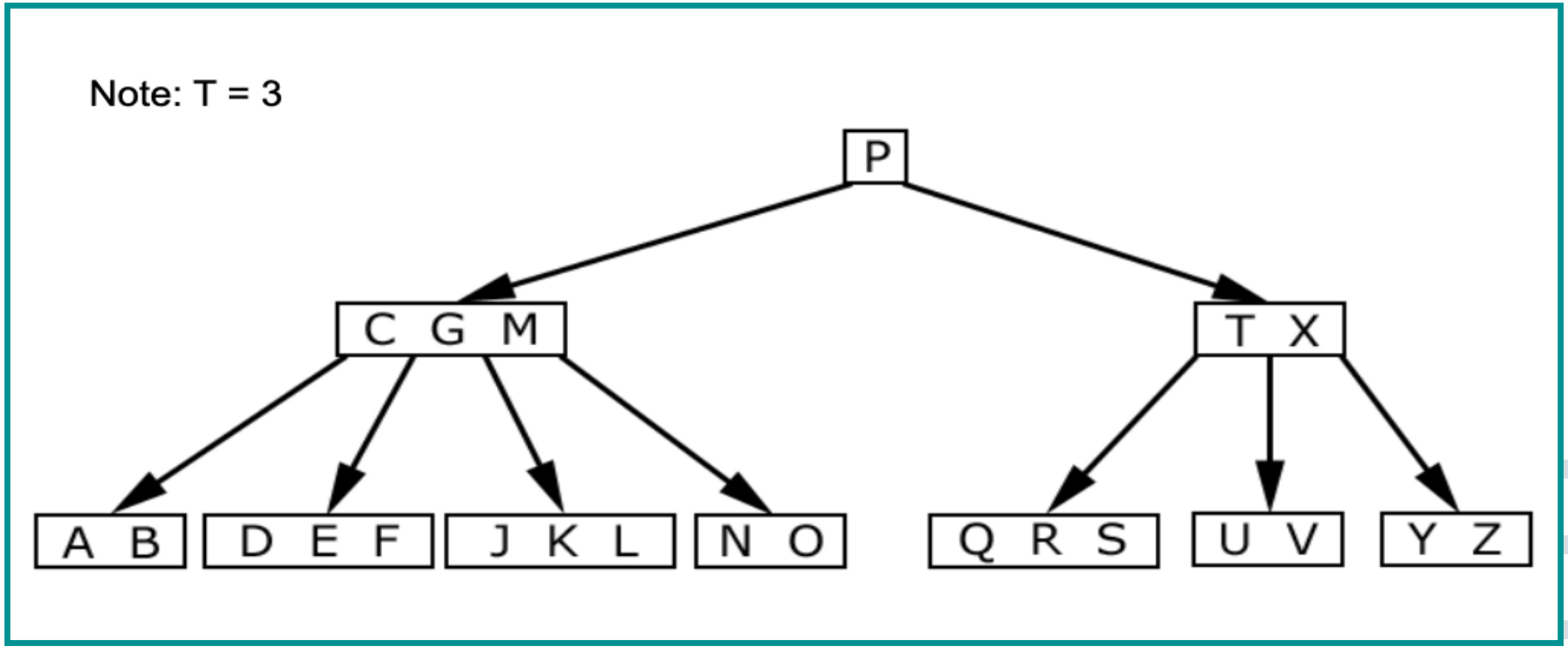
- You can delete a key entry from any node.
- Therefore, you must ensure that before/after deletion, the B-Tree maintains its properties.
- When deleting, you have to ensure that a node doesn't get too small (minimum node size is T – 1). We prevent this by combining nodes together.

Note : B-Tree is defined by the term minimum degree 't'. The value of 't' depends upon disk block size. Every node except the root must contain at least t-1 keys. The root may contain a minimum of 1 key. All nodes (including root) may contain at most (2*t – 1) keys



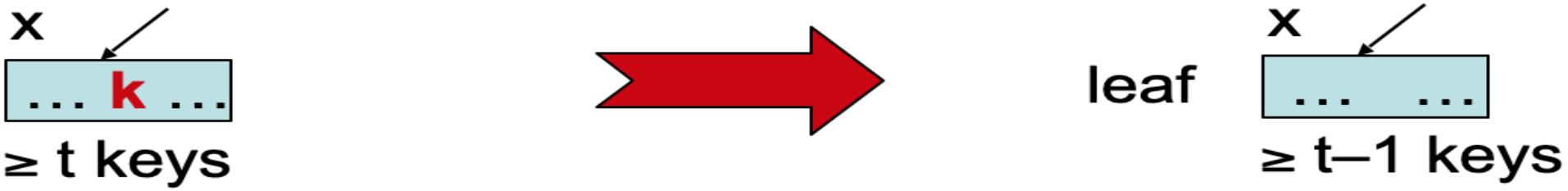
Lets look at an example:

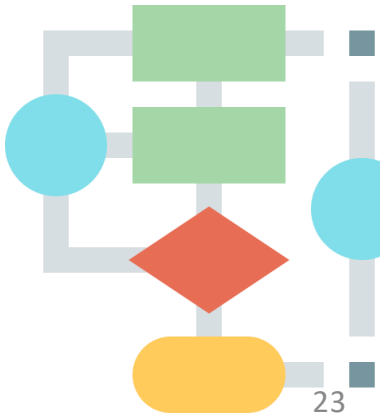
We're given this valid B-Tree , T=3 where T is the minimum degree= number of Childs



Deletion Cases

Case 1: If the key k is in node x and x is a leaf node having at least t keys - then delete k from x.

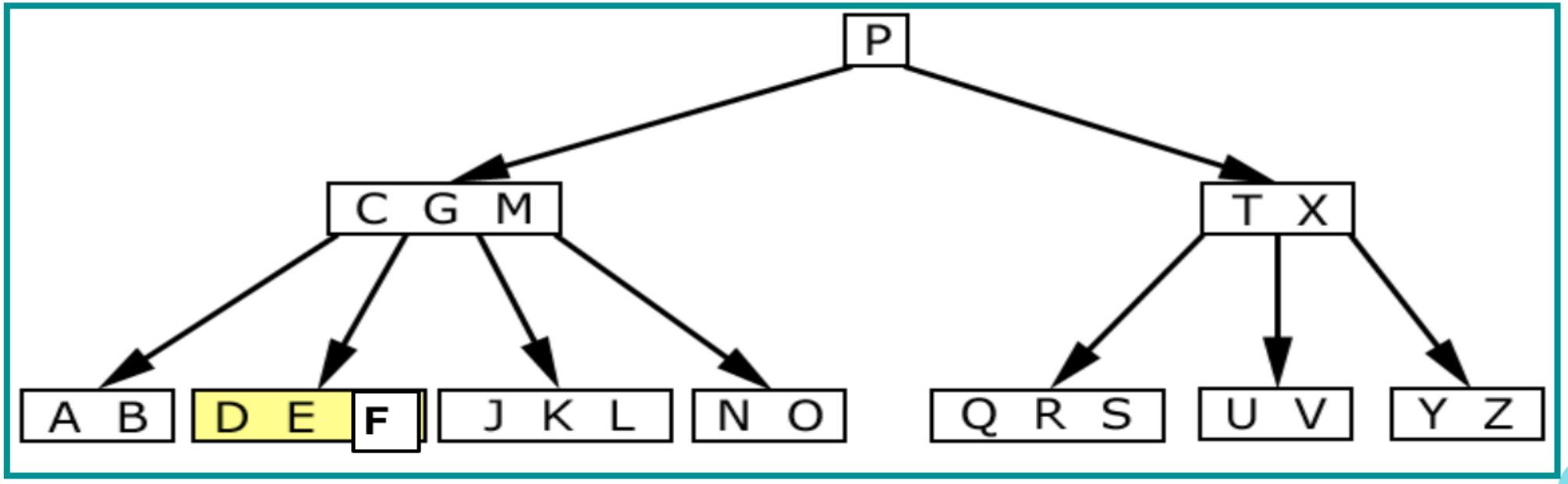




Simple Deletion

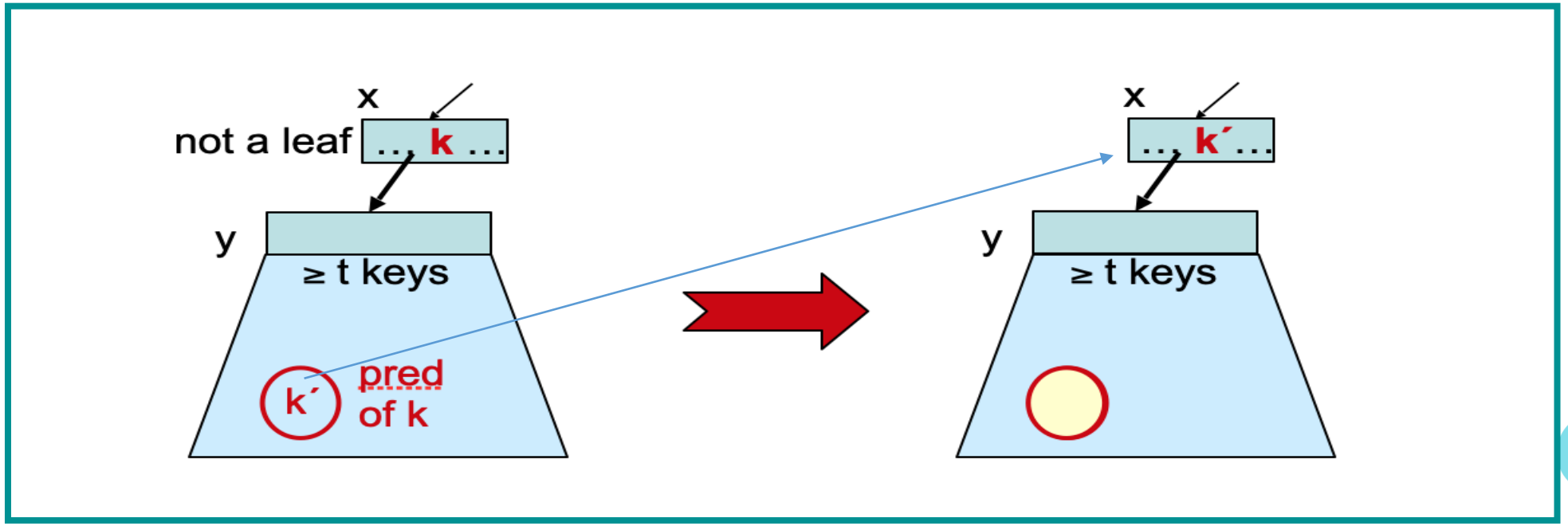
Case 1: We delete "F"

Result: We remove "F" from the leaf node. No further action needed.



Deletion Cases (Continued)

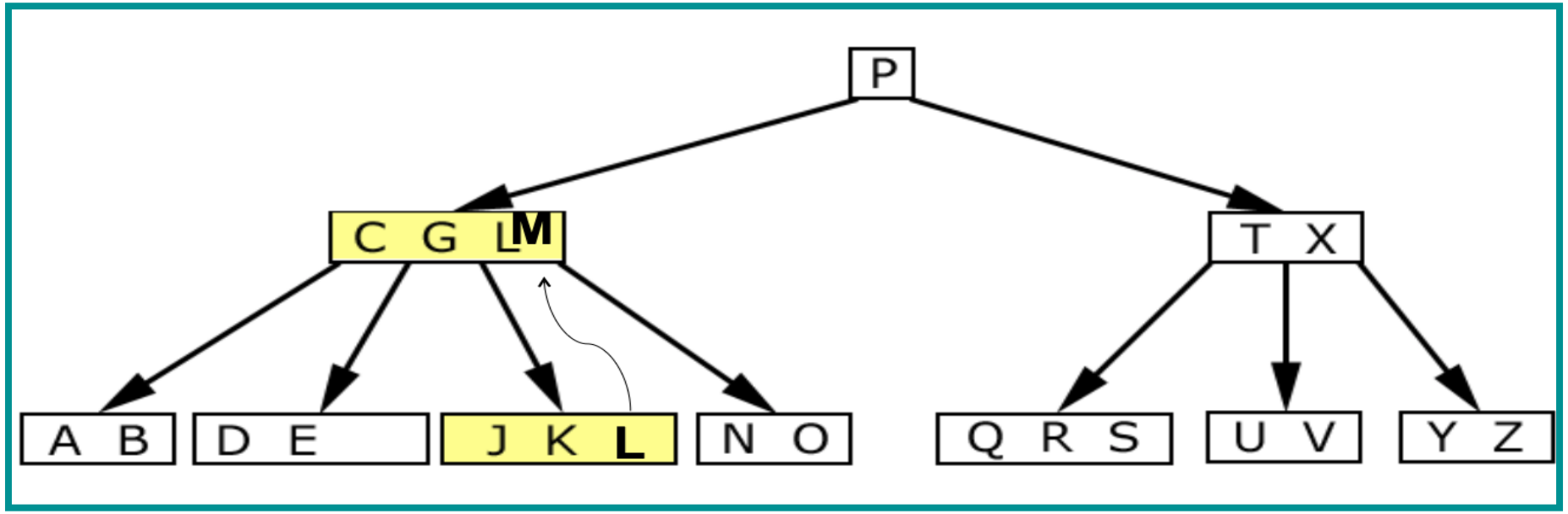
Subcase a: **If the child y that precedes k has at least t keys then find predecessor k´ of k in subtree rooted at y, recursively delete k´ and replace k by k´ in x.**



Deleting and shifting

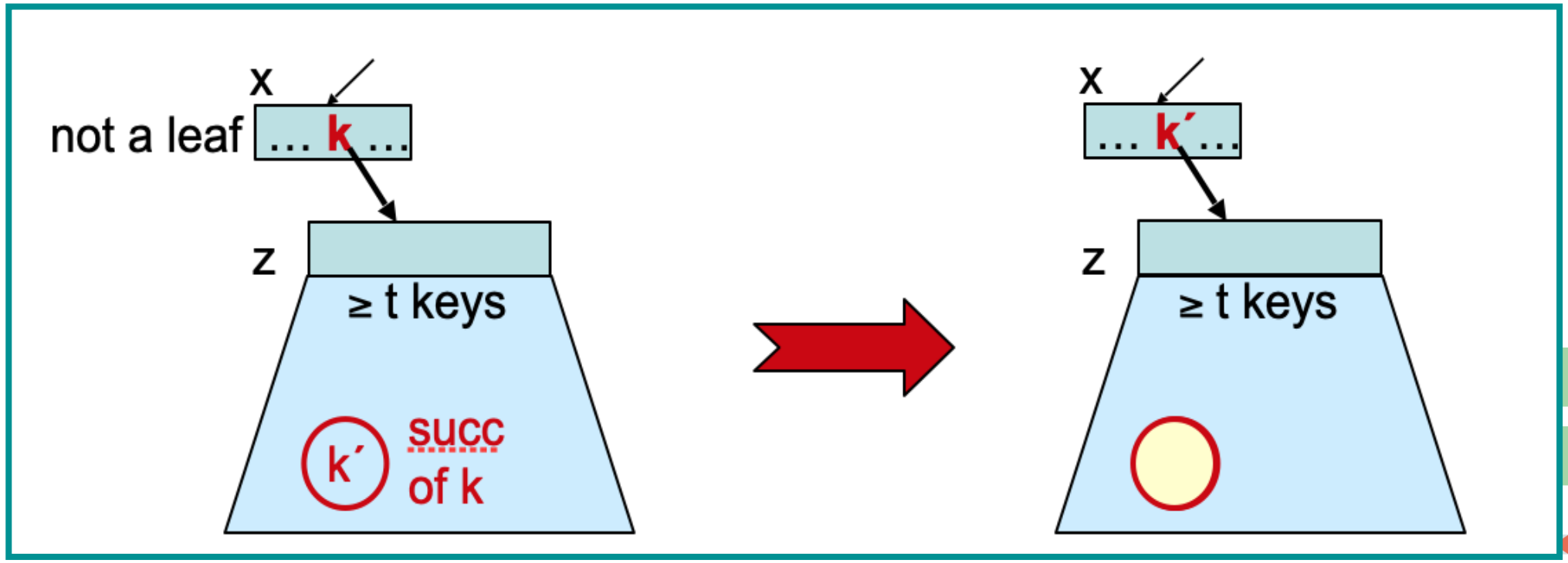
Case 2a: We deleted "M"

Result: We remove "M" from the parent node. Since there are four nodes and two letters, we move "L" to replace "M". Now, the "N O" node has a parent again.



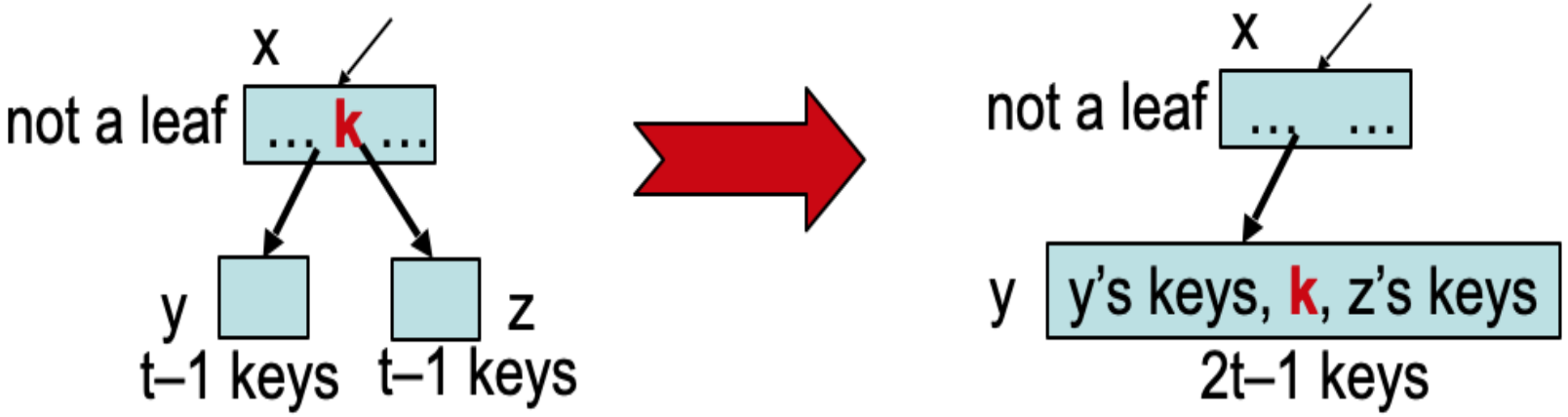
Deletion Cases (Continued)

Subcase B: **Symmetrically, if the child z that follows k in node x has at least t keys then find successor k´ of k in subtree rooted at z, recursively delete k´and replace k by k´ in x.**



Deletion Cases (Continued)

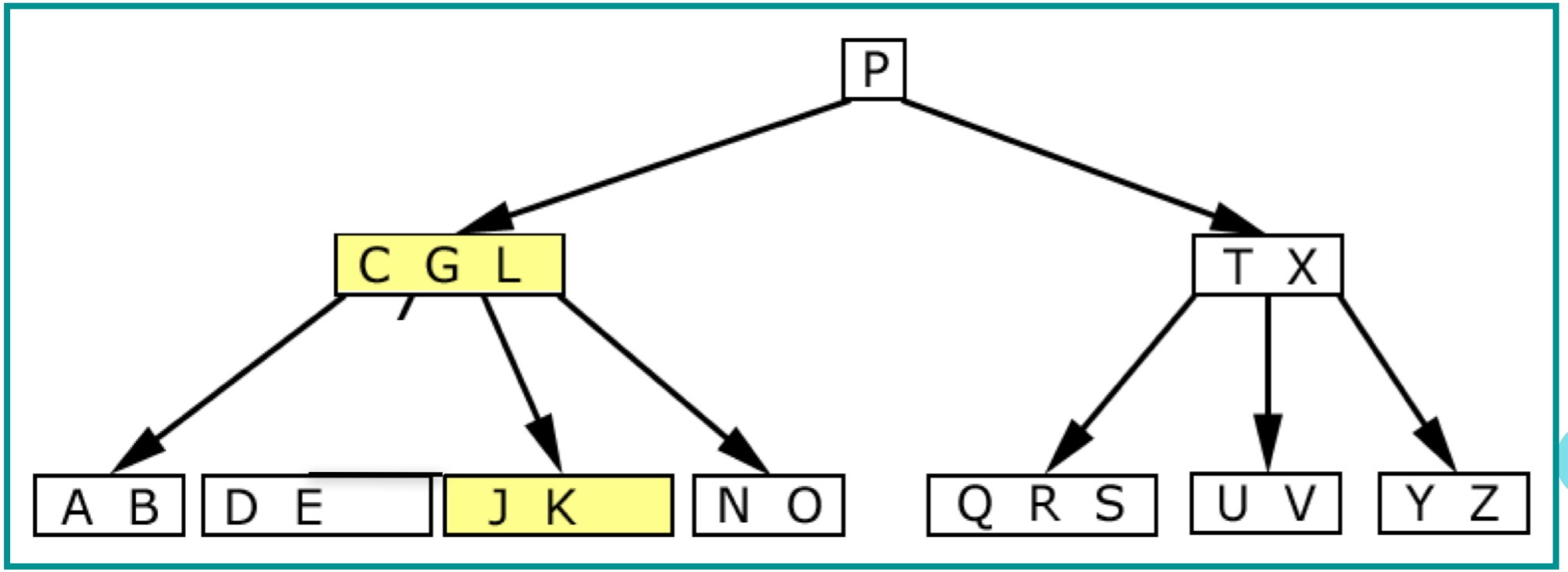
Subcase B: **y and z both have t–1 keys -- merge k and z into y, free z, recursively delete k from y.**



Combining and Deleting

Case 2c: Now, we delete "G"

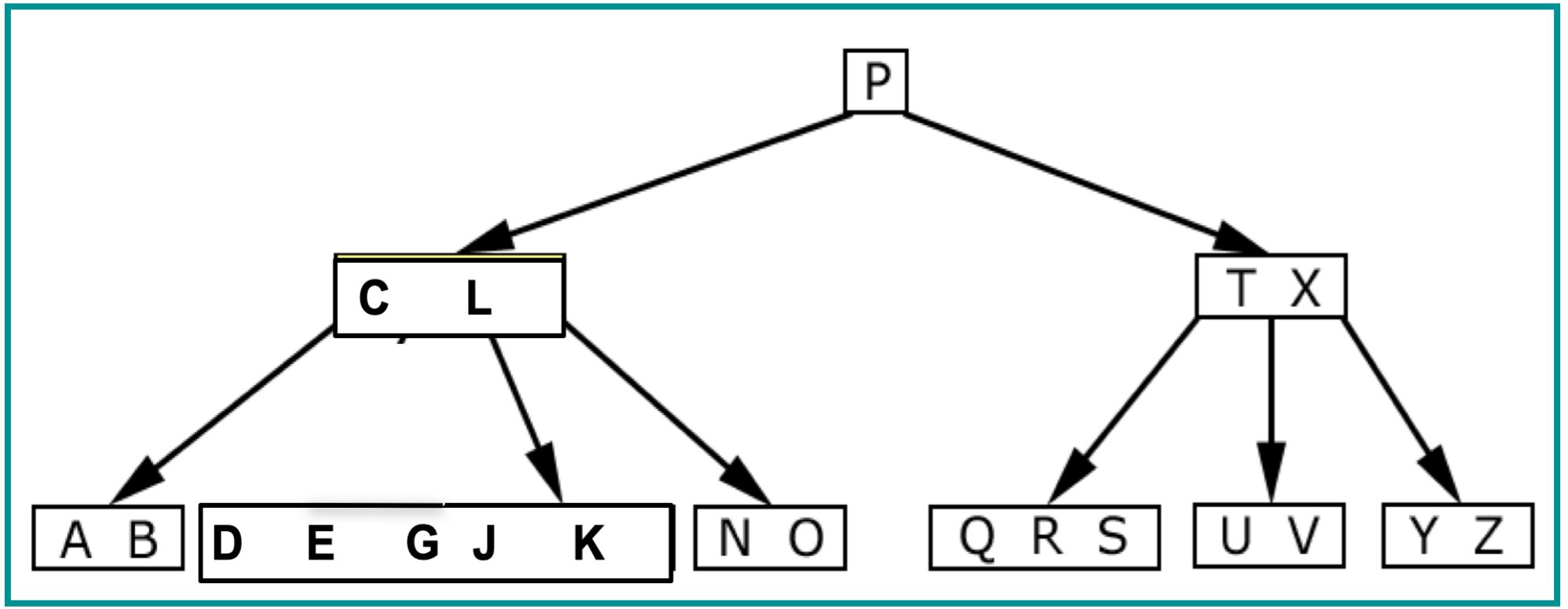
Result: First, we combine nodes "DE" and "JK". Then, we push down "G" into the "DEJK" node and delete it as a leaf.



Combining and Deleting

Case 2c: Now, we delete "G"

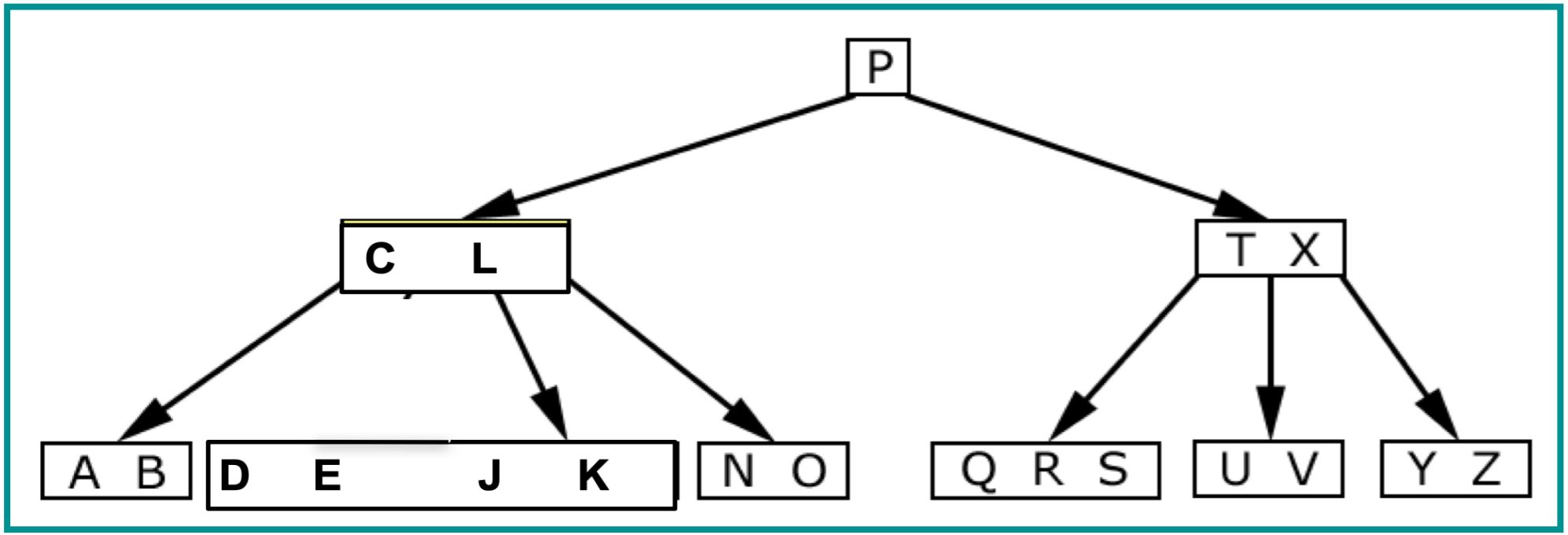
Result: First, we combine nodes "DE" and "JK". Then, we push down "G" into the "DEJK" node and delete it as a leaf.



Combining and Deleting

Case 2c: Now, we delete "G"

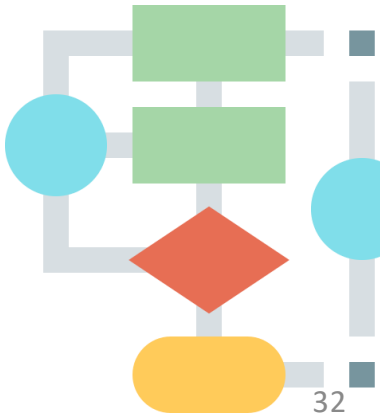
Result: First, we combine nodes "DE" and "JK". Then, we push down "G" into the "DEJK" node and delete it as a leaf.



Deletion Cases (Continued)

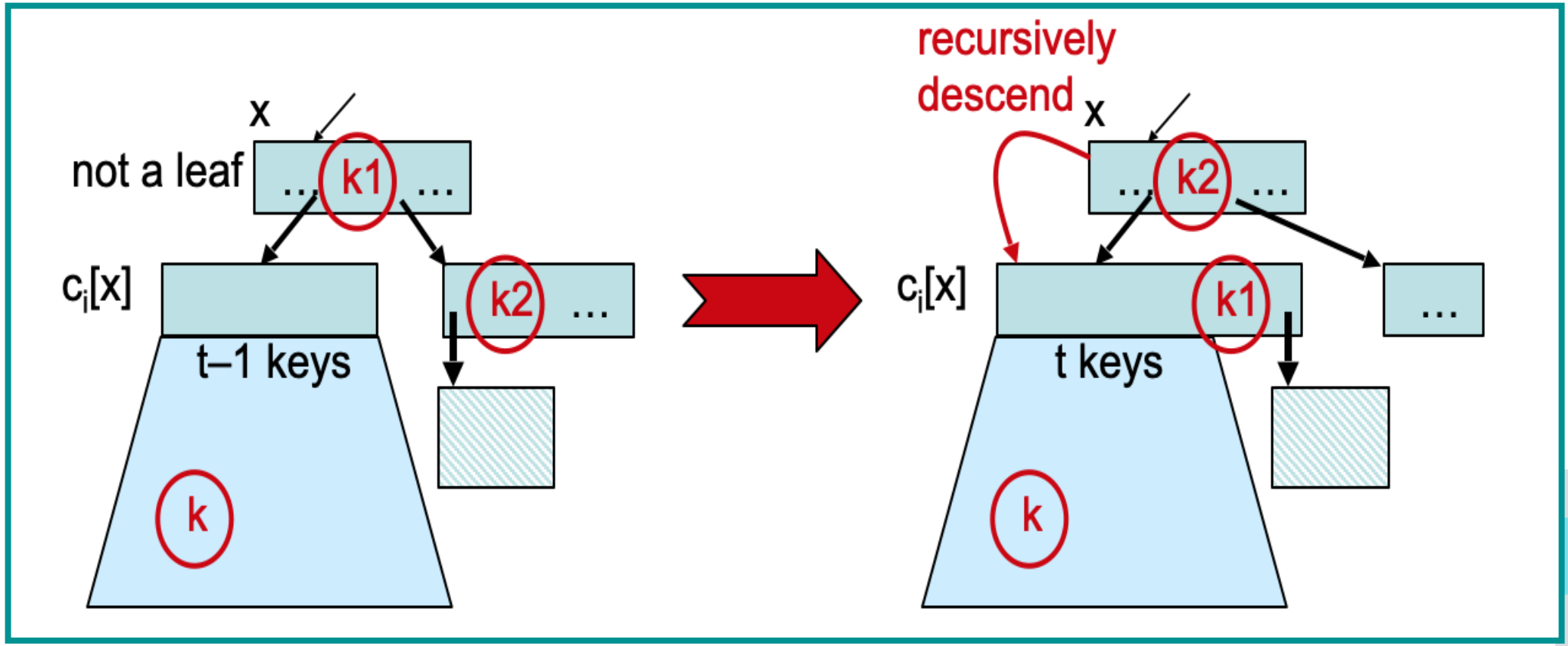
Case 3: k not in internal node. Let ci[x] be the root of t subtree that must contain k, if k is in the tree. If ci[x] has at least t keys, then recursively descend;

otherwise, execute 3.A and 3.B necessary, Refers back to the textbook



Deletion Cases (Continued)

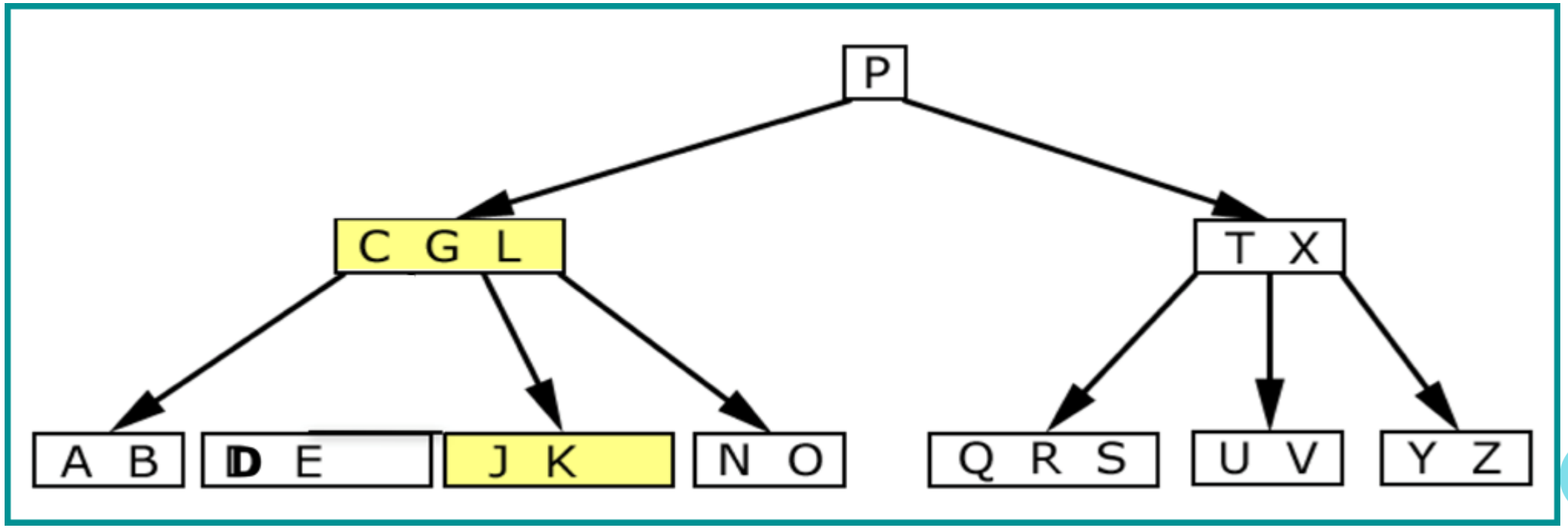
Subcase A: **Subcase A: ci[x] has t–1 keys, and some sibling has at least t keys.**



Combining and Deleting

Case 3b: Now, we delete "D"

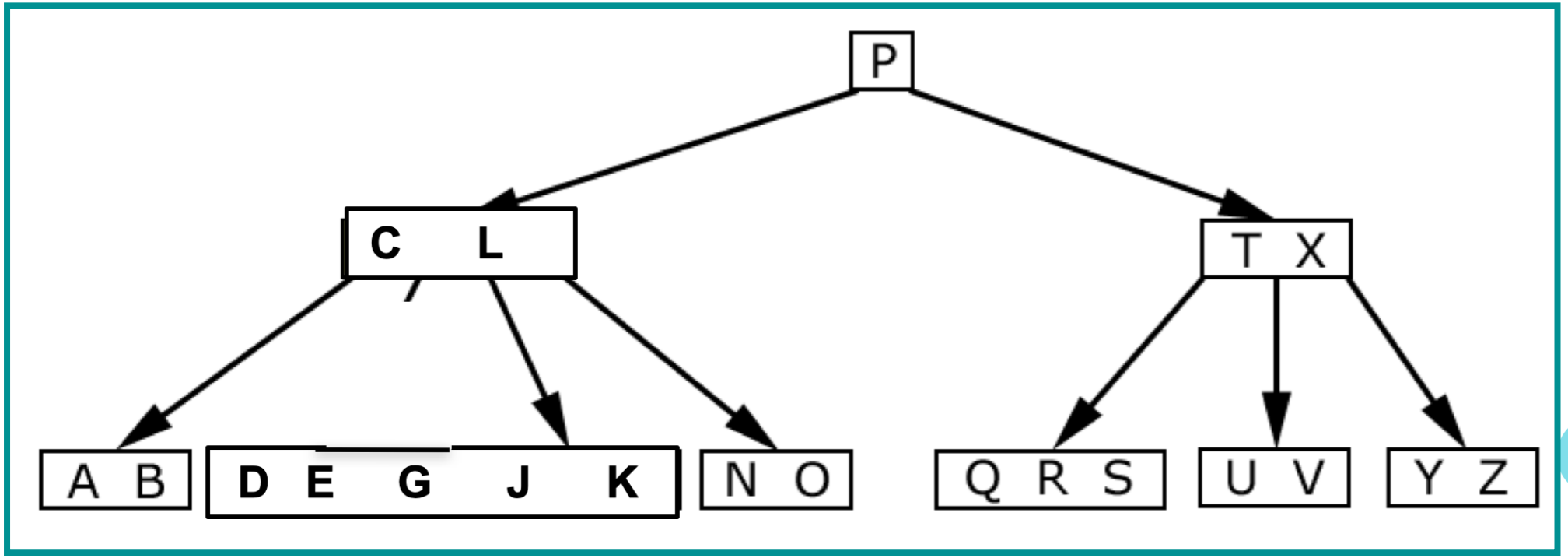
Result: First, we combine nodes "DE" and "JK". Then, we push down "G" into the "DEJK" node and delete "D" as a leaf.



Combining and Deleting

Case 3b: Now, we delete "D"

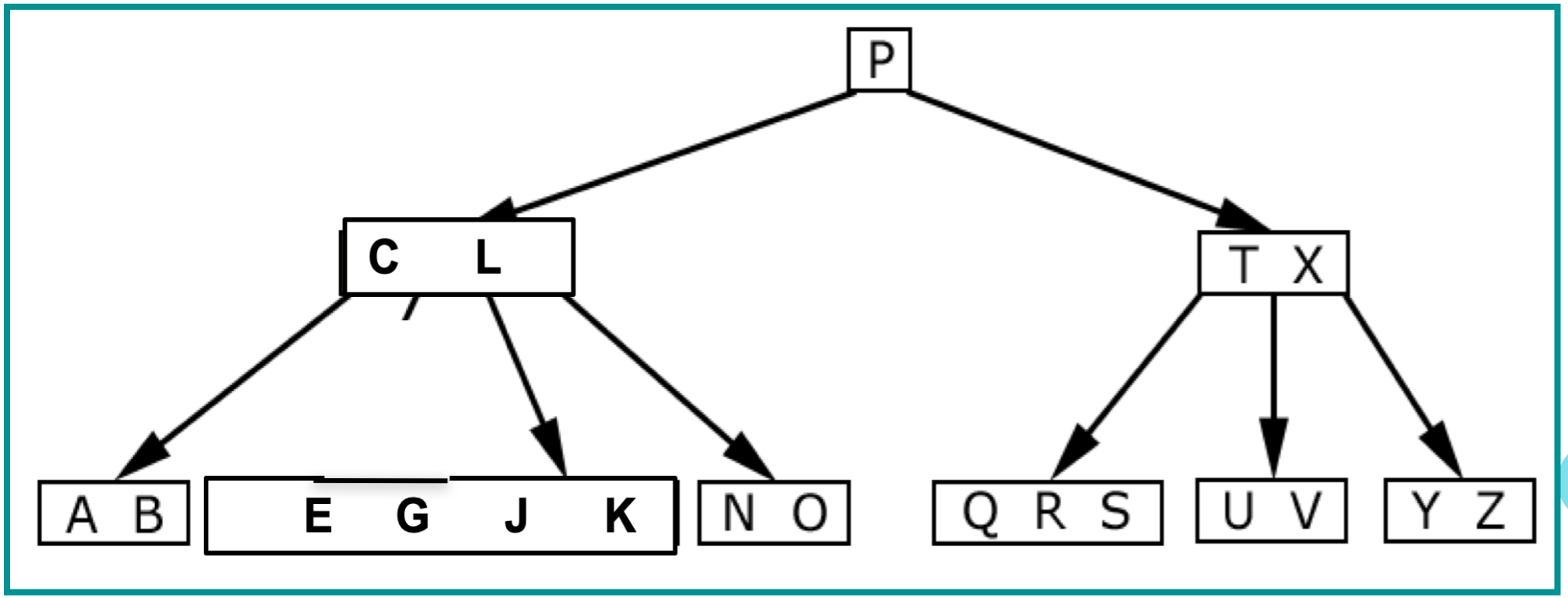
Result: First, we combine nodes "DE" and "JK". Then, we push down "G" into the "DEJK" node and delete "D" as a leaf.



Combining and Deleting

Case 3b: Now, we delete "D"

Result: First, we combine nodes "DE" and "JK". Then, we push down "G" into the "DEJK" node and delete "D" as a leaf.



The End ..

This lecture was from

Introduction_to_Algorithms_Third_Edition_(2009)

chapter 18

